ISE 3434: Applied Optimization

Assignment #4

Due Date: November 5, 2026, 11:59pm

Directions

- Submit one PDF file (your write-up) and one Jupyter notebook (.ipynb) file to Canvas by the due date above. Late submissions are accepted according to the late policy in the syllabus.
- Answer every part of each problem in your PDF, and show your work for the parts done by hand. Handwritten work is acceptable if it is scanned clearly and is easy to read.
- Your notebook should contain working code that prints your results, for Problems 1 and 3.
- Grading considers both the correctness and clarity of your written solutions, and the correctness and readability of your code.

Problem 1: Economic Dispatch (30 points)

A grid operator must decide how much power each of four plants should produce over the next hour, as in the economic dispatch example from lecture. The total output must equal the demand of exactly 600 MW. Running plant i at an output of p_i MW costs $a_i p_i^2 + b_i p_i$ dollars per hour, and each plant has a maximum output, given below.

Plant	a	b	Capacity (MW)
1	0.02	8	350
2	0.04	12	200
3	0.05	15	200
4	0.025	5	150

(a) Formulate the problem as a nonlinear program. Define the decision variables, and write out the objective function and every constraint.

(b) Solve the problem with Gurobi. Report the output of each plant and the total cost per hour. You may start from the power dispatch code in the lecture slides, but make sure your model includes the constraint that total output equals demand.

(c) Suppose instead that each plant simply produces 150 MW. Compute the cost per hour of this plan. How much does the optimal plan from part (b) save per hour?

(d) In the optimal plan, plant 4 runs at its maximum output. The derivative of plant i's cost, $2a_i p_i + b_i$, is approximately the extra cost per hour of producing one more MW at plant i. Compute this derivative for each plant at the optimal plan, and use the results to explain why plant 4 runs at its maximum output.

To submit

- Model description: variables, objective, constraints, and nonnegativity restrictions (part a).
- Your code as part of the single .ipynb file, with results printed.
- A brief written summary of your solution, including your answers to parts (b) through (d).

Problem 2: Unconstrained Optimization (30 points)

Consider the function

$$f(x, y) = x^3 - 12x + y^3 - 3y$$

with no constraints on x and y. Answer the following by hand.

(a) Compute the gradient $\nabla f(x, y)$ and the Hessian $H(x, y)$.

(b) Find all stationary points of f.

(c) Use the Hessian to classify each stationary point as a local minimum, a local maximum, or a saddle point.

(d) Is f convex, concave, or neither? Is the local minimum you found in part (c) a global minimum? Explain.

To submit

- Your answers to parts (a) through (d), with your work, in your PDF. No code is required for this problem.

Problem 3: Gradient Descent for Fitting a Line (40 points)

In lecture, we saw that fitting a line to data is an unconstrained nonlinear program. Fit the line $y = a + bx$ to the following five data points by minimizing the sum of squared errors.

x	y
0	1
1	3
2	4
3	7
4	8

The sum of squared errors, as a function of the intercept a and the slope b, is

$$f(a, b) = (1 - a)^2 + (3 - a - b)^2 + (4 - a - 2b)^2 + (7 - a - 3b)^2 + (8 - a - 4b)^2$$

Use gradient descent, $(a_{k+1}, b_{k+1}) = (a_k, b_k) - \alpha \nabla f(a_k, b_k)$, as in the Gradient Descent worked-example lecture, starting from $(a, b) = (0, 0)$ and stopping when $\|\nabla f\| < 0.01$.

(a) Compute the gradient $\nabla f(a, b)$ by hand, and simplify each partial derivative to the form (number)a + (number)b + (number).

(b) Starting from (0, 0) with learning rate $\alpha = 0.01$, perform two iterations of gradient descent by hand. After each iteration, report the point (a, b), the value of f, and $\|\nabla f\|$.

(c) Implement gradient descent in Python, adapting the code from the Gradient Descent worked-example lecture, with $\alpha = 0.01$. Report the number of iterations until the stopping rule is met, the final values of a and b, and the final value of f. Then check your answer by solving the problem with the Gurobi line-fitting code from lecture.

(d) Repeat part (c) with learning rates $\alpha = 0.005$, 0.02, and 0.035, keeping the cap of 1000 iterations from the lecture code. Report the number of iterations for each, and explain what you observe.

To submit

- Your answers to parts (a) and (b), with your work, in your PDF.
- Your code as part of the single .ipynb file, with results printed.
- A brief written summary of your results for parts (c) and (d).